PROJECT :: Managed Memory with More Concurrency, and Threads

Start Assignment

- Due Wednesday by 11:59pm
- Points 150
- Submitting a file upload
- File Types c, md, jpg, gif, and png
- Available until Dec 5 at 11:59pm

First look at the prompt.
Before you start coding, read the **full assignment prompt (https://csus.instructure.com/courses/21989/pages/project-os-managed-memory-with-more-concurrency-and-threads)** carefully. This includes all sections, examples, and notes. Every detail matters. The description explains how this project builds on the previous one, introduces new concurrency mechanisms, and outlines the expectations for each part. You are responsible for understanding the entire prompt before you begin.

You will **download the provided scaffold file,** `sharedhash.c` **(https://csus.instructure.com/courses/137456/files/26393501?wrap=1)** **(https://csus.instructure.com/courses/137456/files/26393501/download?download_frd=1)** . This is the file you will edit and submit for **Part 1**. Do not rename it, and do not submit multiple files.T he provided scaffold contains all necessary infrastructure, including the full working baseline and shared allocator implementation. Read through the code thoroughly before making any changes.

For **Part 2**, you will create a copy of your working Part 1 solution and rename it `esharedhash.c`. You will modify and experiment with this version only. Both files must be submitted at the end of the project.

Be sure to **read all instructions and comments carefully**. Much of what you need to know is documented directly in the scaffold. Understanding the provided implementation before attempting modifications will save significant time later.

Assignment Revision History

This assignment may be updated during the term. If a newer version is released, it will be clearly labeled (e.g., V002, V003, etc.). Always confirm that you are working from the most recent version before submitting your work. Updates may include clarifications, corrected constants, or improved testing instructions.

Modifying the scaffold.

Keep the constraints tight. The scaffold, Huffman code, printing code, macro definitions, and the mmap pool size are all fixed. Only `umalloc` and `ufree` may be altered, plus the portions of first-fit logic they depend on. The output hash must remain identical, so structural behavior visible to the test harness cannot change. That includes allocation order, block layout, and any side effects that feed into the hash. Treat any difference in the hash as a regression.

Test allocation failures against the unmodified scaffold first; if the baseline succeeds, then your modified version must also succeed. You cannot compensate by expanding the pool or tweaking global definitions. Keep modifications minimal and defensible, and explain in the writeup why each change is necessary for correctness rather than convenience. The justification is graded more heavily than the code itself.

Images in Markdown Files

Some students were having difficulty in getting images to reliably show up in markdown files. First, I didn't expect you to include images, however, if you decide to do so, and you are concerned about them showing up, then you may now just upload the images as a part of the submission.

Current Revision:

- **V002: Update to allowed changes and images.**
- **V001: Initial release – expect minor revisions**

Assignment Outline

1. **Review the Full Prompt**
   Read every section in the project description carefully. The project has two major parts:
   - **Part 1:** Convert the provided shared-memory multi-process implementation into a threaded version.
   - **Part 2:** Experiment with allocator optimizations and concurrency improvements.
     Each part builds directly on the last, so do not move ahead until your previous step is fully working.
2. **Download and Inspect the Scaffold**
   - Download `sharedhash.c` **(https://csus.instructure.com/courses/137456/files/26393501?wrap=1)** **(https://csus.instructure.com/courses/137456/files/26393501/download?download_frd=1)** from Canvas.
   - Read through the entire file before editing.
   - Do not modify any code outside the **STUDENT SECTION** unless explicitly instructed.
   - Familiarize yourself with the shared allocator, semaphore usage, and process handling.

3. **Part 1: Convert to a Threaded Implementation**
   Replace all process-related logic with threads using the **pthread** API. Each worker thread should process one data block concurrently. You will:
   - Replace `fork()` and `waitpid()` with `pthread_create()` and `pthread_join()`.
   - Replace all process synchronization using semaphores with mutexes.
   - Remove pipes and instead store each thread's result in a shared array.
   - Ensure all hash results remain consistent with the original multi-process version.
   - Confirm that your program compiles and runs correctly with multiple threads.
4. **Part 2: Explore and Optimize Concurrency**
   Copy your working `sharedhash.c` to `esharedhash.c`. Modify this version only. In this part, you will:
   - Experiment with ways to reduce lock contention in the allocator.
   - Implement one or more optimizations such as per-thread memory pools, fine-grained locking, or opportunistic allocation strategies.
   - Measure and compare performance between your baseline (`sharedhash`) and optimized (`esharedhash`) versions.
   - Submit both programs along with a short report describing your changes and observations.
5. **Test Frequently**
   Compile and test after each significant change using small and large input files. Use timing comparisons and correctness checks to verify progress. Example commands:

   ```
   gcc -pthread -o sharedhash sharedhash.c
   gcc -pthread -o esharedhash esharedhash.c ./sharedhash test_1MB.bin -t ./esharedhash test_1MB.bin -t
   ```

   Use tools like **Valgrind** to check for memory errors and leaks during development.
6. **Validate and Prepare for Submission**
   - Remove any debugging prints before submission.
   - Ensure your programs compile cleanly under Linux with `gcc -Wall -Wextra -pthread`.
   - Verify that both versions produce identical final hashes for the same inputs.
   - Include your performance report (plain text or markdown).
   - Submit both `sharedhash.c` and `esharedhash.c`.
   - Submit your markdown files with the required analysis

Reminder:

Each assignment version is designed to stand on its own. You are encouraged to treat this as a portfolio-quality project — document your work clearly and make sure it demonstrates your understanding of concurrency, synchronization, and performance optimization.